

HANDS_ON_Logging

Hands-on: Logging & Debugging in Apache NiFi

Apache NiFi 2.8.0 · UI: <https://localhost:8443/nifi> · Login `admin` `N=/Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0`

NiFi-Logging hat **vier Ebenen**, die wir hier durchgehen: 1. **Log-Dateien** auf der Platte (`logs/`) 2. **In-UI-Werkzeuge**: Bulletins, Data Provenance, Status History 3. **Eigene Logs erzeugen** mit den Prozessoren `LogAttribute` / `LogMessage` 4. **Log-Konfiguration** (`conf/logback.xml`, Log-Level pro Komponente)

1. Die Log-Dateien : `nifi-2.8.0/logs/`

Das ist die wichtigste Quelle bei “warum schlägt mein Flow fehl”.

Datei	Inhalt	Wann reinschauen
<code>nifi-app.log</code>	Haupt-Log: Prozessor-Fehler, Stacktraces, Start/Stop, Repository	Immer zuerst
<code>nifi-user.log</code>	Authentifizierung/ Autorisierung: Logins, abgelehnte Zugriffe	Login-/Security-Probleme
<code>nifi-bootstrap.log</code>	JVM-Start, Speicher, Prozess-Bootstrap	NiFi startet gar nicht
<code>nifi-request.log</code>	Jeder HTTP-Request an die Web-API	API-/UI-Debugging
<code>nifi-deprecation.log</code>	Nutzung veralteter Features	Vor Versions-Upgrade

Rotation: Die aktuelle Datei ist immer `nifi-app.log` (ohne Datum). Stündlich wird nach `nifi-app_YYYY-MM-TT_HH.0.log` archiviert. Steuerung in `logback.xml` (rollingPolicy, maxHistory).

Aufbau einer Logzeile:

```
2026-05-31 22:14:07,331 ERROR [Timer-Driven Process Thread-3]
o.a.n.p.standard.PutDatabaseRecord PutDatabaseRecord[id=...] Failed to process ...
  ^Zeitstempel      ^Level    ^Thread                                ^Logger (Prozessor-
Klasse)            ^Nachricht
```

Befehle für die Schulung:

```
tail -f $N/logs/nifi-app.log          # live mitlesen
grep -i error $N/logs/nifi-app.log | tail -20    # letzte Fehler
```

```
grep "PutDatabaseRecord" $N/logs/nifi-app.log # nach Prozessor filtern
grep -A 30 "Failed to process" $N/logs/nifi-app.log # Stacktrace mit Kontext
```

2. Logging in der UI : die drei Werkzeuge

a) Bulletins : der schnelle Blick

- **Am Prozessor:** Ein rotes/gelbes Quadrat oben rechts an der Box. Maus drüber → Fehlermeldung direkt am Knoten (die UI-Kurzfassung dessen, was in `nifi-app.log` steht).
- **Global:** Hamburger-Menü ☰ (oben rechts) → **Bulletin Board** : alle Bulletins zentral.
- Bulletins zeigen die **letzten ~5 Minuten** an WARN/ERROR. Älteres nur noch im Log.
- Pro Prozessor lässt sich das **Bulletin Level** einstellen (Configure → Settings → "Bulletin Level").

b) Data Provenance : "was geschah mit MEINER FlowFile"

NiFis stärkstes Debug-Tool : kein Text-Log, sondern die komplette Lineage jeder einzelnen Datei. - Rechtsklick auf Prozessor → **View data provenance** (oder global ☰ → **Data Provenance**). - Liste der Events: RECEIVE, FETCH, ROUTE, ATTRIBUTES_MODIFIED, CONTENT_MODIFIED, SEND, DROP ... - Auf das ⓘ/Lupe-Symbol einer Zeile klicken → **Details / Attributes / Content**: - **Attributes**: alle FlowFile-Attribute zu diesem Zeitpunkt (mit "vorher/nachher"). - **Content**: Inhalt der FlowFile vor und nach dem Prozessor (View/Download). - **Lineage (Graph)**: rechts unten "Show Lineage" → kompletter Weg der FlowFile als Graph, abspielbar über die Zeitachse.

c) Status History : Durchsatz/Queue über Zeit

- Rechtsklick auf Prozessor (oder Connection/Process Group) → **View status history**.
- Graphen über In/Out, Tasks, durchschnittliche Dauer, Queue-Größe : gut für Performance-Fragen.

3. Lab L : Logging & Debugging (fertig auf der Canvas)

Process Group "**Lab L - Logging & Debugging**" mit zwei Ketten:

```
Kette 1 (Erfolg, INFO):
  GetFile —> UpdateAttribute —> LogAttribute (INFO) —> PutFile-OK
                (setzt pipeline,                (schreibt alle Attribute
                geprueft_am)                      nach nifi-app.log)

Kette 2 (Fehler, ERROR):
  GetFile (Fehler-Demo) —> PutDatabaseRecord (Tabelle "gibt_es_nicht")
                                ↳failure/retry—> LogAttribute (ERROR-Pfad, Log
Payload=true)
```

Übung A : LogAttribute auf dem Erfolgspfad

1. PG "Lab L" starten (Rechtsklick → Start).

2. Datei einwerfen:

```
cp $N/data/samples/orders.csv $N/data/input/labL/
```

3. **Ergebnis:**

- Datei landet in \$N/data/output/labL_ok/
- In nifi-app.log erscheint ein Block von LogAttribute mit allen Attributen, u.a. filename, uuid, pipeline = logging-demo, geprueft_am = ...:

```
grep -A 20 "LogAttribute" $N/logs/nifi-app.log | tail -30
```

So macht man den unsichtbaren FlowFile-Zustand sichtbar : **das ist printf-Debugging für NiFi.**

Übung B : Fehler sichtbar loggen statt still verwerfen

1. Datei in den **Fehler-Ordner** werfen:

```
cp $N/data/samples/orders.csv $N/data/input/labL_err/
```

2. PutDatabaseRecord scheitert (Tabelle gibt_es_nicht) → FlowFile geht über failure an LogAttribute (ERROR-Pfad).

3. **Ergebnis:**

- In der UI: **rotes Bulletin** an PutDatabaseRecord (Maus drüber zeigt den DB-Fehler).
- In nifi-app.log: ein ERROR-Eintrag von PutDatabaseRecord **plus** der LogAttribute-Block (mit Inhalt, weil Log Payload = true).

```
grep -i "error" $N/logs/nifi-app.log | tail -10
```

4. **Lehrpunkt:** Statt failure zu auto-terminieren (Datei verschwindet lautlos) hängt man einen LogAttribute/LogMessage oder PutFile an : so ist jeder Fehler nachvollziehbar.

Übung C : Data Provenance lesen

1. Nach Übung A: Rechtsklick auf **LogAttribute (INFO)** → **View data provenance.**
2. Jüngstes Event öffnen (📄) → Tab **Attributes**: zeige pipeline und geprueft_am.
3. Tab **Content** → "View": der CSV-Inhalt der FlowFile.
4. "Show Lineage" → der Graph zeigt GetFile → UpdateAttribute → LogAttribute → PutFile.

Reset

```
find $N/data/input/labL $N/data/input/labL_err $N/data/output/labL_ok -type f -delete  
# PG "Lab L" stoppen
```

4. LogAttribute / LogMessage : die wichtigsten Properties

LogAttribute (loggt Attribute, optional Inhalt): | Property | Bedeutung | Empfehlung | |—| —|—| | Log Level | trace/debug/info/warn/error | info (Demo), error für Fehlerpfad | | Log FlowFile Properties | Standard-Attribute (filename, uuid, size ...) mitloggen | true | | Attributes to Log (regex) | nur bestimmte Attribute, z.B. pipeline|geprueft_am | leer = alle | | Log Payload | FlowFile-**Inhalt** mitloggen (Property heißt Log Payload, nicht “Log Body”) | true nur für kleine Dateien/Debug | | Output Format | Line per Attribute oder Single Line | Line per Attribute (lesbarer) | | Log prefix | Markertext vor dem Eintrag, z.B. >>> FEHLER | optional, erleichtert grep |

LogMessage loggt eine **freie Nachricht** (mit Expression Language), ohne alle Attribute: - Log Level = info/error - Log message = z.B.

Datei \${filename} (\${filesize} Bytes) verarbeitet um \${now()}

Tipp: LogMessage für gezielte Meldungen (“Job X fertig”), LogAttribute für vollständiges Attribut-Dump beim Debuggen.

5. Log-Level ändern : conf/logback.xml

Steuert, **was** geloggt wird. Beispiel: einen Prozessor auf DEBUG (mehr Details):

```
<logger name="org.apache.nifi.processors.standard.PutDatabaseRecord" level="DEBUG"/>
```

Beispiel: die lauten Repository-Meldungen leiser stellen:

```
<logger name="org.apache.nifi.controller.repository" level="WARN"/>
```

- NiFi lädt logback.xml **automatisch nach ~30 Sekunden neu : kein Neustart nötig.**
- Logger-Namen = Java-Paket/Klasse. Den passenden Namen findet man in jeder Logzeile (Spalte “Logger”, z.B. o.a.n.p.standard.PutDatabaseRecord → ausgeschrieben org.apache.nifi.processors.standard.PutDatabaseRecord).

Schnell-Diagnose-Reihenfolge (für Teilnehmer)

1. **Bulletin** am Prozessor anschauen (Maus drüber) : 80 % der Fälle sofort klar.
2. Reicht nicht → nifi-app.log per grep -i error / tail -f.
3. “Wo blieb meine Datei” → **Data Provenance** + Lineage-Graph.
4. Tiefer graben → in logback.xml den betroffenen Prozessor auf **DEBUG**, 30s warten, erneut testen.
5. Beim Bauen: failure-Relationships **nicht blind auto-terminieren** : an LogAttribute / LogMessage oder PutFile hängen, damit Fehler sichtbar bleiben.